L37: LLMs for Program Analysis: Test Generation

Software Analysis

IIT Guwahati · Nov 3, 2026

Warm-up Quiz (1/6)

What does this C++ program print?

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

bool is_nonzero(int x) { return x != 0; }

bool all_nonzero(const std::vector<int>& v) {
  return std::all_of(v.begin(), v.end(),
                     is_nonzero);
}

int main() {
  std::cout << std::boolalpha
            << all_nonzero({}) << ", "
            << all_nonzero({0}) << ", "
            << all_nonzero({1}) << "\n";
}
```

☐ `false, false, true` ☐ `true, false, true` ☐ `true, true, true`

Warm-up Quiz (2/6)

What does this C++ program print?

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

bool is_nonzero(int x) { return x != 0; }

bool all_nonzero(const std::vector<int>& v) {
  return std::all_of(v.begin(), v.end(),
                     is_nonzero);
}

int main() {
  std::cout << std::boolalpha
            << all_nonzero({}) << ", "
            << all_nonzero({0}) << ", "
            << all_nonzero({1}) << "\n";
}
```

☐ `false, false, true` ☒ `true, false, true` ☐ `true, true, true`

Warm-up Quiz (3/6)

What does this C++ program print?

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

bool is_nonzero(int x) { return x != 0; }

bool all_nonzero(const std::vector<int>& v) {
  return std::all_of(v.begin(), v.end(),
                     is_nonzero);
}

int main() {
  std::cout << std::boolalpha
            << all_nonzero({}) << ", "
            << all_nonzero({0}) << ", "
            << all_nonzero({1}) << "\n";
}
```

← `all_of`: true unless some element fails the test

☐ `false, false, true` ☒ `true, false, true` ☐ `true, true, true`

Warm-up Quiz (4/6)

What does this C++ program print?

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

bool is_nonzero(int x) { return x != 0; }

bool all_nonzero(const std::vector<int>& v) {
  return std::all_of(v.begin(), v.end(),
                     is_nonzero);
}

int main() {
  std::cout << std::boolalpha
            << all_nonzero({}) << ", "
            << all_nonzero({0}) << ", "
            << all_nonzero({1}) << "\n";
}
```

←empty: nothing fails ⇝ `true`

☐ `false, false, true` ☒ `true, false, true` ☐ `true, true, true`

Warm-up Quiz (5/6)

What does this C++ program print?

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

bool is_nonzero(int x) { return x != 0; }

bool all_nonzero(const std::vector<int>& v) {
  return std::all_of(v.begin(), v.end(),
                     is_nonzero);
}

int main() {
  std::cout << std::boolalpha
            << all_nonzero({}) << ", "
            << all_nonzero({0}) << ", "
            << all_nonzero({1}) << "\n";
}
```

←0 fails ⇝ `false`

☐ `false, false, true` ☒ `true, false, true` ☐ `true, true, true`

Warm-up Quiz (6/6)

What does this C++ program print?

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

bool is_nonzero(int x) { return x != 0; }

bool all_nonzero(const std::vector<int>& v) {
  return std::all_of(v.begin(), v.end(),
                     is_nonzero);
}

int main() {
  std::cout << std::boolalpha
            << all_nonzero({}) << ", "
            << all_nonzero({0}) << ", "
            << all_nonzero({1}) << "\n";
}
```

←1 passes ⇝ true

☐ `false, false, true` ☒ `true, false, true` ☐ `true, true, true`

Motivation (1/6)

Limitations of the techniques we have seen: random testing, fuzzing, symbolic and concolic execution

```
Config cfg = Config::load(path);
Db db(cfg.url(), cfg.user());
Cart cart(db, user_id);
cart.add(item, 2);
assert(cart.total() == 40);
```

a realistic unit test

Motivation (2/6)

Limitations of the techniques we have seen: random testing, fuzzing, symbolic and concolic execution

- Building **realistic objects**
  e.g. a valid configuration, a connected database

```
Config cfg = Config::load(path);
Db db(cfg.url(), cfg.user());
Cart cart(db, user_id);
cart.add(item, 2);
assert(cart.total() == 40);
```

a realistic unit test

a fuzzer or solver must invent a valid `cfg`, a reachable `db`, and the expected 40

Motivation (3/6)

Limitations of the techniques we have seen: random testing, fuzzing, symbolic and concolic execution

- Building **realistic objects**
  e.g. a valid configuration, a connected database
- Using **complex APIs** in the right order

```
Config cfg = Config::load(path);
Db db(cfg.url(), cfg.user());
Cart cart(db, user_id);
cart.add(item, 2);
assert(cart.total() == 40);
```

a realistic unit test

a fuzzer or solver must invent a valid `cfg`, a reachable `db`, and the expected 40

Motivation (4/6)

Limitations of the techniques we have seen: random testing, fuzzing, symbolic and concolic execution

- Building **realistic objects**
  e.g. a valid configuration, a connected database
- Using **complex APIs** in the right order
- Setting up **mocks** for the environment

```
Config cfg = Config::load(path);
Db db(cfg.url(), cfg.user());
Cart cart(db, user_id);
cart.add(item, 2);
assert(cart.total() == 40);
```

a realistic unit test

a fuzzer or solver must invent a valid `cfg`, a reachable `db`, and the expected 40

Motivation (5/6)

Limitations of the techniques we have seen: random testing, fuzzing, symbolic and concolic execution

- Building **realistic objects**
  e.g. a valid configuration, a connected database
- Using **complex APIs** in the right order
- Setting up **mocks** for the environment
- Understanding **natural language**
  documentation, issue reports, comments

```
Config cfg = Config::load(path);
Db db(cfg.url(), cfg.user());
Cart cart(db, user_id);
cart.add(item, 2);
assert(cart.total() == 40);
```

a realistic unit test

a fuzzer or solver must invent a valid `cfg`, a reachable `db`, and the expected 40

Motivation (6/6)

Limitations of the techniques we have seen: random testing, fuzzing, symbolic and concolic execution

- Building **realistic objects**
  e.g. a valid configuration, a connected database
- Using **complex APIs** in the right order
- Setting up **mocks** for the environment
- Understanding **natural language**
  documentation, issue reports, comments
- Every tool relies on **hand-designed heuristics**

```
Config cfg = Config::load(path);
Db db(cfg.url(), cfg.user());
Cart cart(db, user_id);
cart.add(item, 2);
assert(cart.total() == 40);
```

a realistic unit test

a fuzzer or solver must invent a valid `cfg`, a reachable `db`, and the expected 40

What Is a Large Language Model? (1/5)

A model that, given a sequence of **tokens**, predicts a **probability** for each possible next token

`assert` `(` `cart` `.` `total` `()` `==`

the **prompt**: tokens so far

What Is a Large Language Model? (2/5)

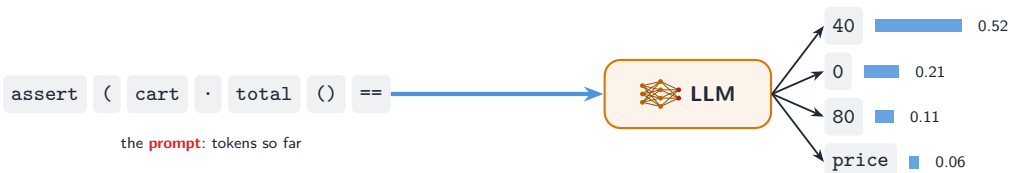
A model that, given a sequence of **tokens**, predicts a **probability** for each possible next token



the **prompt**: tokens so far

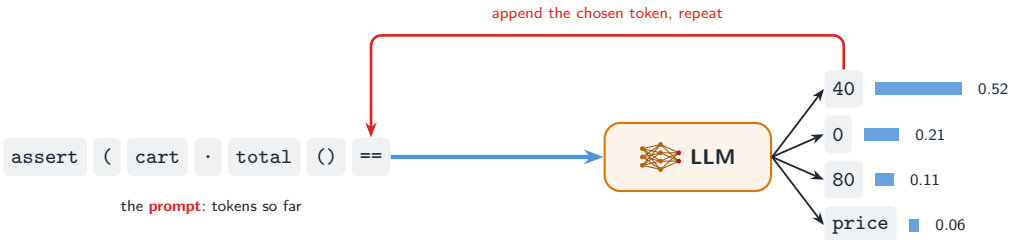
What Is a Large Language Model? (3/5)

A model that, given a sequence of **tokens**, predicts a **probability** for each possible next token



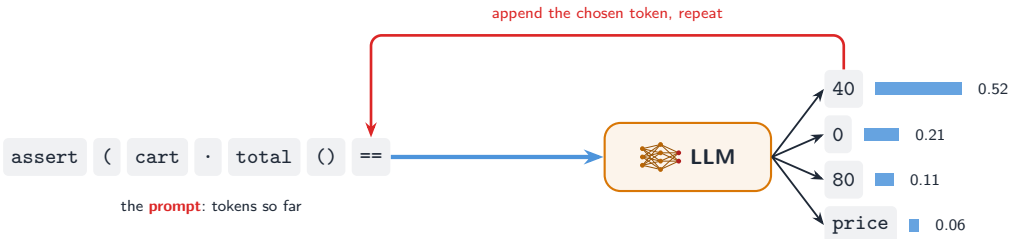
What Is a Large Language Model? (4/5)

A model that, given a sequence of **tokens**, predicts a **probability** for each possible next token



What Is a Large Language Model? (5/5)

A model that, given a sequence of **tokens**, predicts a **probability** for each possible next token



Trained on huge amounts of code and text: it has seen many tests, docs and APIs

Prompts, Context Windows and Hallucination (1/5)

- **Prompt**: the input text; **completion**: the generated text

Prompts, Context Windows and Hallucination (2/5)

- **Prompt**: the input text; **completion**: the generated text
- **Context window**: the maximum prompt length
  a whole code base does not fit: we must *choose* what to show

Prompts, Context Windows and Hallucination (3/5)

- **Prompt**: the input text; **completion**: the generated text
- **Context window**: the maximum prompt length
  a whole code base does not fit: we must *choose* what to show
- **Sampling**: the same prompt can give different completions
  a *temperature* controls how random the choice of token is

Prompts, Context Windows and Hallucination (4/5)

- **Prompt**: the input text; **completion**: the generated text
- **Context window**: the maximum prompt length
  a whole code base does not fit: we must *choose* what to show
- **Sampling**: the same prompt can give different completions
  a *temperature* controls how random the choice of token is
- **Hallucination**: fluent output that is wrong
  e.g. calling a function that does not exist

Prompts, Context Windows and Hallucination (5/5)

- **Prompt**: the input text; **completion**: the generated text
- **Context window**: the maximum prompt length
  a whole code base does not fit: we must *choose* what to show
- **Sampling**: the same prompt can give different completions
  a *temperature* controls how random the choice of token is
- **Hallucination**: fluent output that is wrong
  e.g. calling a function that does not exist

LLM output is a **guess**: we must give it the right context, and **check** it

Pradel, Program Analysis, Stuttgart: LLM-based Test Generation; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Neural versus Symbolic Reasoning (1/3)

Neural (LLMs)

- Reasons about **fuzzy** inputs: natural language, naming conventions
- Gives **probabilistic** answers
- Hard to understand and debug

Neural versus Symbolic Reasoning (2/3)

Neural (LLMs)

- Reasons about **fuzzy** inputs: natural language, naming conventions
- Gives **probabilistic** answers
- Hard to understand and debug

Symbolic (program analysis)

- Reasons about **facts**: types, data flow, paths
- Can give **guarantees**
- Harder to build, easier to debug

Neural versus Symbolic Reasoning (3/3)

Neural (LLMs)

- Reasons about **fuzzy** inputs: natural language, naming conventions
- Gives **probabilistic** answers
- Hard to understand and debug

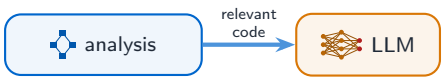
Symbolic (program analysis)

- Reasons about **facts**: types, data flow, paths
- Can give **guarantees**
- Harder to build, easier to debug

Complementary strengths: **combine** them. This is **neuro-symbolic** program analysis.

Combining LLMs and Program Analysis (1/4)

1. analysis as context provider

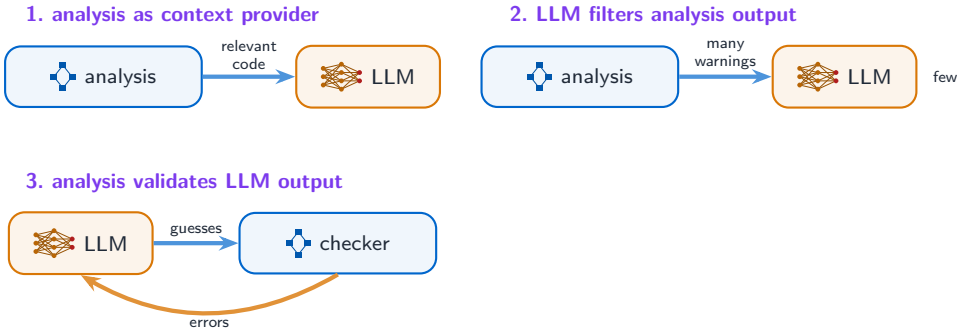


Combining LLMs and Program Analysis (2/4)



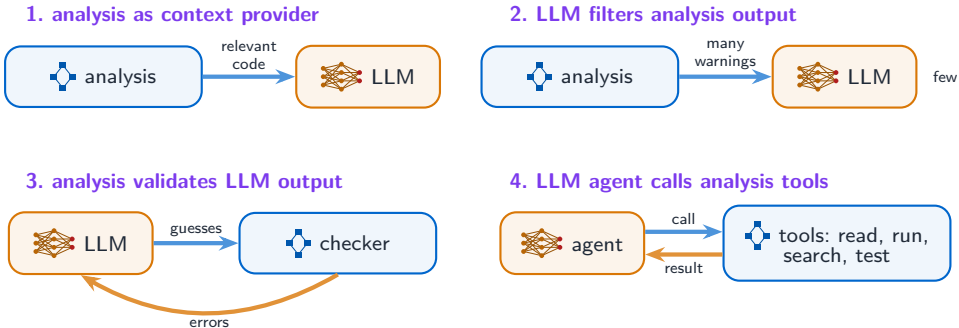
Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Combining LLMs and Program Analysis (3/4)



Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Combining LLMs and Program Analysis (4/4)



Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Pattern 1: Analysis as Context (1/6)

The whole code base does not fit in the prompt. Analysis selects **what matters**.
Task: write a unit test for `price`.

```
1  struct Row { int unit_price; };
2  struct Db { Row lookup(int id) const; };
3  int discount(int qty) {
4    if (qty >= 10) return 15;
5    return 0;
6  }

8  int price(const Db& db, int id, int qty) {
9    Row r = db.lookup(id);
10   int base = r.unit_price * qty;
11   int d = discount(qty);
12   return base - base * d / 100;
13 }
14 void handle(const Request& req, const Db& db) {
15   int id = std::stoi(req.param("id")); // untrusted
16   std::cout « price(db, id, 3);
17 }
```

Analysis	Gives the LLM	In our example
symbol table	names in scope	`Db::lookup`, `Row::unit_price`: how to set up a `Db` and a `Row`

Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Pattern 1: Analysis as Context (2/6)

The whole code base does not fit in the prompt. Analysis selects **what matters**.
Task: write a unit test for `price`.

```
1  struct Row { int unit_price; };
2  struct Db { Row lookup(int id) const; };
3  int discount(int qty) {
4    if (qty >= 10) return 15;
5    return 0;
6  }

8  int price(const Db& db, int id, int qty) {
9    Row r = db.lookup(id);
10   int base = r.unit_price * qty;
11   int d = discount(qty);
12   return base - base * d / 100;
13 }
14 void handle(const Request& req, const Db& db) {
15   int id = std::stoi(req.param("id")); // untrusted
16   std::cout « price(db, id, 3);
17 }
```

Analysis	Gives the LLM	In our example
symbol table	names in scope	`Db::lookup`, `Row::unit_price`: how to set up a `Db` and a `Row`
types	argument and return types	`price(const Db&, int, int) -> int`

Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Pattern 1: Analysis as Context (3/6)

The whole code base does not fit in the prompt. Analysis selects **what matters**.
Task: write a unit test for `price`.

```
1  struct Row { int unit_price; };
2  struct Db { Row lookup(int id) const; };
3  int discount(int qty) {
4    if (qty >= 10) return 15;
5    return 0;
6  }

8  int price(const Db& db, int id, int qty) {
9    Row r = db.lookup(id);
10   int base = r.unit_price * qty;
11   int d = discount(qty);
12   return base - base * d / 100;
13 }
14 void handle(const Request& req, const Db& db) {
15   int id = std::stoi(req.param("id")); // untrusted
16   std::cout « price(db, id, 3);
17 }
```

Analysis	Gives the LLM	In our example
symbol table	names in scope	`Db::lookup`, `Row::unit_price`: how to set up a `Db` and a `Row`
types	argument and return types	`price(const Db&, int, int) -> int`
call graph	callers, callees	callees `lookup`, `discount`; caller `handle`

Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Pattern 1: Analysis as Context (4/6)

The whole code base does not fit in the prompt. Analysis selects **what matters**.
Task: write a unit test for `price`.

```
1  struct Row { int unit_price; };
2  struct Db { Row lookup(int id) const; };
3  int discount(int qty) {
4    if (qty >= 10) return 15;
5    return 0;
6  }

8  int price(const Db& db, int id, int qty) {
9    Row r = db.lookup(id);
10   int base = r.unit_price * qty;
11   int d = discount(qty);
12   return base - base * d / 100;
13 }
14 void handle(const Request& req, const Db& db) {
15   int id = std::stoi(req.param("id")); // untrusted
16   std::cout « price(db, id, 3);
17 }
```

Analysis	Gives the LLM	In our example
symbol table	names in scope	`Db::lookup`, `Row::unit_price`: how to set up a `Db` and a `Row`
types	argument and return types	`price(const Db&, int, int) -> int`
call graph	callers, callees	callees `lookup`, `discount`; caller `handle`
program slice	lines that affect a line	return on 12: lines 9–12, 4–5

Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Pattern 1: Analysis as Context (5/6)

The whole code base does not fit in the prompt. Analysis selects **what matters**.
Task: write a unit test for `price`.

```
1  struct Row { int unit_price; };
2  struct Db { Row lookup(int id) const; };
3  int discount(int qty) {
4    if (qty >= 10) return 15;
5    return 0;
6  }

8  int price(const Db& db, int id, int qty) {
9    Row r = db.lookup(id);
10   int base = r.unit_price * qty;
11   int d = discount(qty);
12   return base - base * d / 100;
13 }
14 void handle(const Request& req, const Db& db) {
15   int id = std::stoi(req.param("id")); // untrusted
16   std::cout « price(db, id, 3);
17 }
```

Analysis	Gives the LLM	In our example
symbol table	names in scope	`Db::lookup`, `Row::unit_price`: how to set up a `Db` and a `Row`
types	argument and return types	`price(const Db&, int, int) -> int`
call graph	callers, callees	callees `lookup`, `discount`; caller `handle`
program slice	lines that affect a line	return on 12: lines 9–12, 4–5
taint path	source-to-sink flow	`req.param` (15) → `db.lookup` (9)

Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Pattern 1: Analysis as Context (6/6)

The whole code base does not fit in the prompt. Analysis selects **what matters**. Task: write a unit test for `price`.

```
1 struct Row { int unit_price; };
2 struct Db { Row lookup(int id) const; };
3 int discount(int qty) {
4   if (qty >= 10) return 15;
5   return 0;
6 }

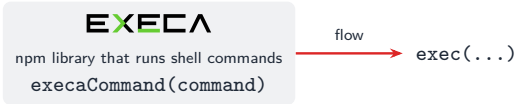
8 int price(const Db& db, int id, int qty) {
9   Row r = db.lookup(id);
10  int base = r.unit_price * qty;
11  int d = discount(qty);
12  return base - base * d / 100;
13 }
14 void handle(const Request& req, const Db& db) {
15  int id = std::stoi(req.param("id")); // untrusted
16  std::cout « price(db, id, 3);
17 }
```

Analysis	Gives the LLM	In our example
symbol table	names in scope	`Db::lookup`, `Row::unit_price`: how to set up a `Db` and a `Row`
types	argument and return types	`price(const Db&, int, int) -> int`
call graph	callers, callees	callees `lookup`, `discount`; caller `handle`
program slice	lines that affect a line	return on 12: lines 9–12, 4–5
taint path	source-to-sink flow	`req.param` (15) → `db.lookup` (9)
coverage	lines no test reaches	line 4 `return 15`: ask for qty ≥ 10

Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Pattern 2: Filtering Analysis Outputs (1/5)

Static taint analysis finds flows from an API to exec. Is each flow a vulnerability?



Chow, Schäfer, Pradel, ISSTA 2023; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Pattern 2: Filtering Analysis Outputs (2/5)

Static taint analysis finds flows from an API to exec. Is each flow a vulnerability?



Chow, Schäfer, Pradel, ISSTA 2023; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

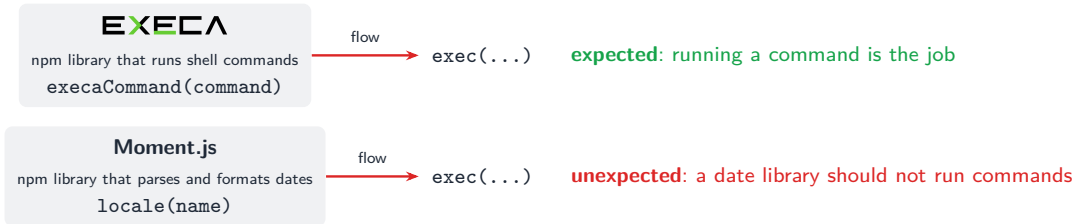
Pattern 2: Filtering Analysis Outputs (3/5)

Static taint analysis finds flows from an API to exec. Is each flow a vulnerability?



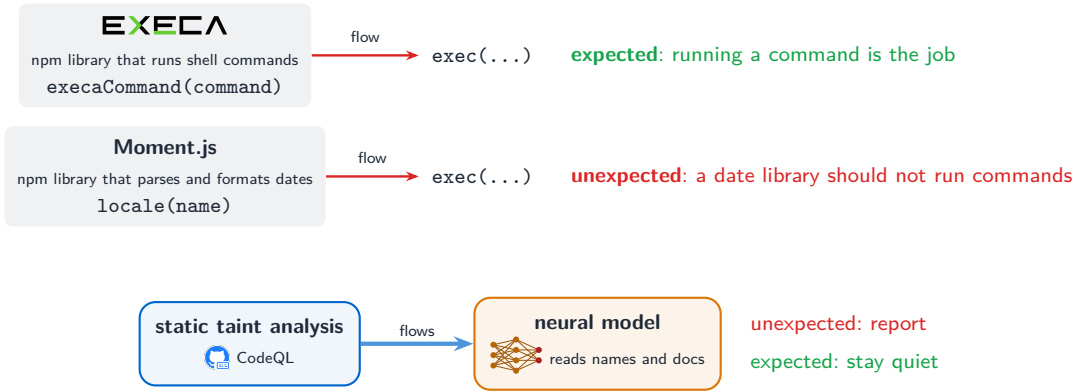
Pattern 2: Filtering Analysis Outputs (4/5)

Static taint analysis finds flows from an API to exec. Is each flow a vulnerability?



Pattern 2: Filtering Analysis Outputs (5/5)

Static taint analysis finds flows from an API to exec. Is each flow a vulnerability?



Chow, Schäfer, Pradel, ISSTA 2023; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Pattern 3: Symbolic Validation (1/5)

Goal: add missing type annotations to Python code. A neural model **predicts**; a type checker **validates**.

input: no annotations

```python
def find_match(color):
  """
  Args:
    color (str): color to match on
  """
  candidates = get_colors()
  for candidate in candidates:
    if color == candidate:
      return color
  return None

def get_colors():
  return ["red", "blue", "green"]
```

Slot	Top-3 predictions
`color`	int, str, bool
return	str, Optional[str], None
`get_colors`	List[str], List[Any], str

Pradel, Gousios, Liu, Chandra, ESEC/FSE 2020; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Pattern 3: Symbolic Validation (2/5)

Goal: add missing type annotations to Python code. A neural model **predicts**; a type checker **validates**.

input: no annotations

```python
def find_match(color):
  """
  Args:
    color (str): color to match on
  """
  candidates = get_colors()
  for candidate in candidates:
    if color == candidate:
      return color
  return None

def get_colors():
  return ["red", "blue", "green"]
```

Slot	Top-3 predictions
`color`	int, str, bool
return	str, Optional[str], None
`get_colors`	List[str], List[Any], str

Pradel, Gousios, Liu, Chandra, ESEC/FSE 2020; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Pattern 3: Symbolic Validation (3/5)

Goal: add missing type annotations to Python code. A neural model **predicts**; a type checker **validates**.

input: no annotations

```python
def find_match(color):
  """
  Args:
    color (str): color to match on
  """
  candidates = get_colors()
  for candidate in candidates:
    if color == candidate:
      return color
  return None

def get_colors():
  return ["red", "blue", "green"]
```

Slot	Top-3 predictions
color	int, str, bool
return	str, Optional[str], None
get_colors	List[str], List[Any], str

Top-1 everywhere: the checker reports **type errors**

Pradel, Gousios, Liu, Chandra, ESEC/FSE 2020; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Pattern 3: Symbolic Validation (4/5)

Goal: add missing type annotations to Python code. A neural model **predicts**; a type checker **validates**.

input: no annotations

```python
def find_match(color):
  """
  Args:
    color (str): color to match on
  """
  candidates = get_colors()
  for candidate in candidates:
    if color == candidate:
      return color
  return None

def get_colors():
  return ["red", "blue", "green"]
```

Slot	Top-3 predictions
color	int, **str**, bool
return	str, **Optional[str]**, None
get_colors	**List[str]**, List[Any], str

Top-1 everywhere: the checker reports **type errors**

Search the predictions until the checker is satisfied: **consistent types**

Pradel, Gousios, Liu, Chandra, ESEC/FSE 2020; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

Pattern 3: Symbolic Validation (5/5)

Goal: add missing type annotations to Python code. A neural model **predicts**; a type checker **validates**.

output: annotated

```python
def find_match(color: str) -> Optional[str]:
  """
  Args:
    color (str): color to match on
  """
  candidates = get_colors()
  for candidate in candidates:
    if color == candidate:
      return color
  return None

def get_colors() -> List[str]:
  return ["red", "blue", "green"]
```

Slot	Top-3 predictions
color	int, **str**, bool
return	str, **Optional[str]**, None
get_colors	**List[str]**, List[Any], str

Top-1 everywhere: the checker reports **type errors**

Search the predictions until the checker is satisfied: **consistent types**

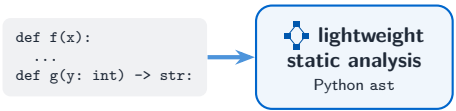
Output: the code with the types written in, plus the `typing` imports

Pradel, Gousios, Liu, Chandra, ESEC/FSE 2020; Pradel, Program Analysis, Stuttgart: Neuro-Symbolic Program Analysis

TypeWriter: Workflow (1/5)

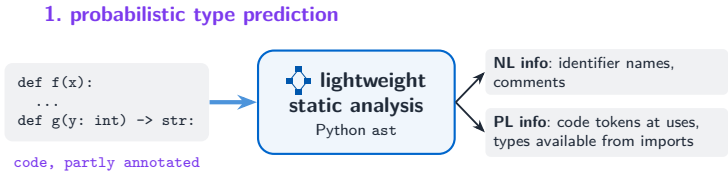
An example of **symbolic validation** (Pattern 3)

1. probabilistic type prediction



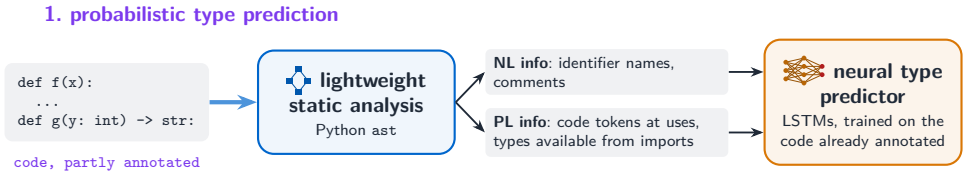
TypeWriter: Workflow (2/5)

An example of **symbolic validation** (Pattern 3)



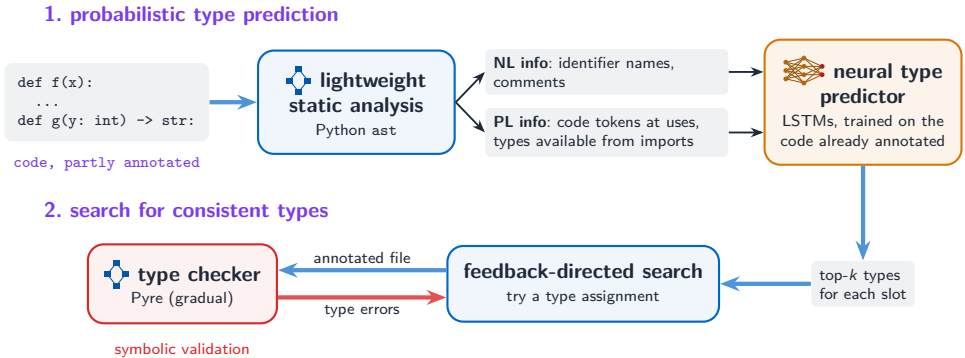
TypeWriter: Workflow (3/5)

An example of **symbolic validation** (Pattern 3)



TypeWriter: Workflow (4/5)

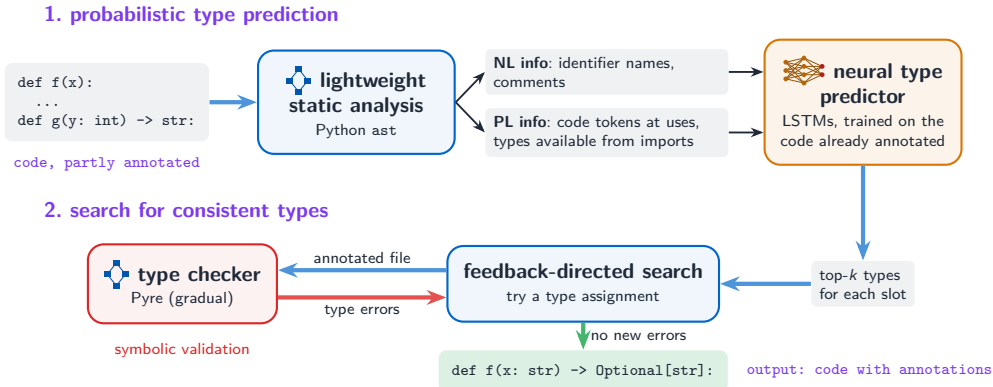
An example of **symbolic validation** (Pattern 3)



Pradel, Gousios, Liu, Chandra, ESEC/FSE 2020

TypeWriter: Workflow (5/5)

An example of **symbolic validation** (Pattern 3)



Pradel, Gousios, Liu, Chandra, ESEC/FSE 2020

Quiz: Which Pattern? (1/4)

Which of the four patterns does each tool use?

1. A linter reports 500 warnings; an LLM ranks the 10 most likely real bugs

2. An LLM writes C code; the code is kept only if it compiles

3. The prompt contains the callers of the function under test

Quiz: Which Pattern? (2/4)

Which of the four patterns does each tool use?

1. A linter reports 500 warnings; an LLM ranks the 10 most likely real bugs — `filter (2)`

2. An LLM writes C code; the code is kept only if it compiles

3. The prompt contains the callers of the function under test

Quiz: Which Pattern? (3/4)

Which of the four patterns does each tool use?

1. A linter reports 500 warnings; an LLM ranks the 10 most likely real bugs — `filter (2)`

2. An LLM writes C code; the code is kept only if it compiles — `validate (3)`

3. The prompt contains the callers of the function under test —

Quiz: Which Pattern? (4/4)

Which of the four patterns does each tool use?

1. A linter reports 500 warnings; an LLM ranks the 10 most likely real bugs — `filter (2)`

2. An LLM writes C code; the code is kept only if it compiles — `validate (3)`

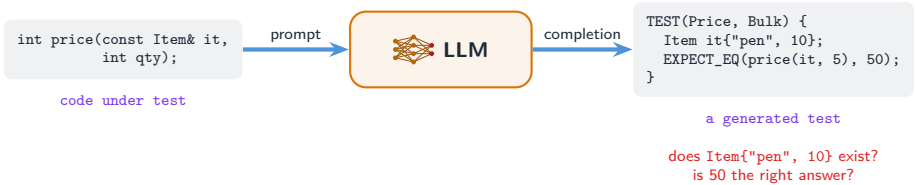
3. The prompt contains the callers of the function under test — `context (1)`

LLMs as Test Generators (1/3)



Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

LLMs as Test Generators (2/3)



Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

LLMs as Test Generators (3/3)



Problems of the naive approach

- the LLM **guesses** input types and constructors
- it invents **mocks** that differ from the real setup
- it guesses the **expected value**

Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Prompt Ingredients (1/6)

The prompt decides how effective an LLM-based tool is



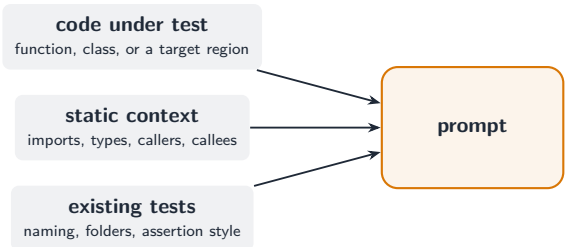
Prompt Ingredients (2/6)

The prompt decides how effective an LLM-based tool is



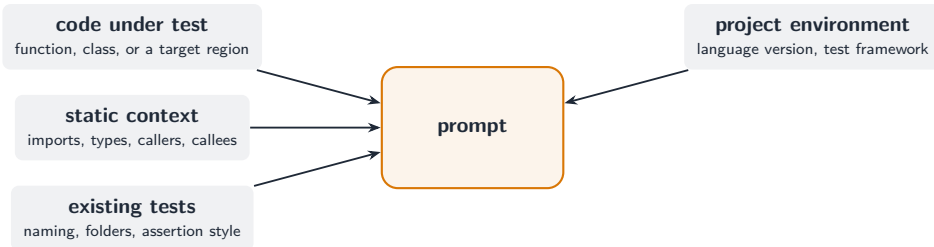
Prompt Ingredients (3/6)

The prompt decides how effective an LLM-based tool is



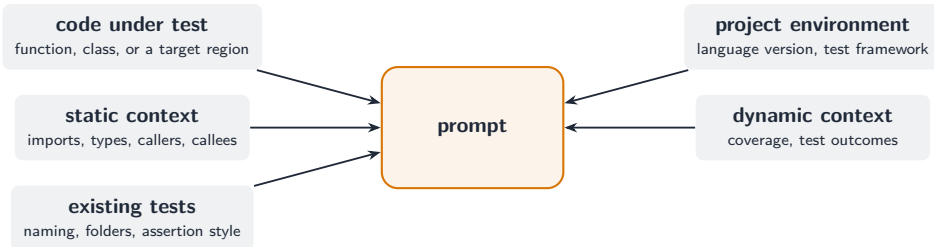
Prompt Ingredients (4/6)

The prompt decides how effective an LLM-based tool is



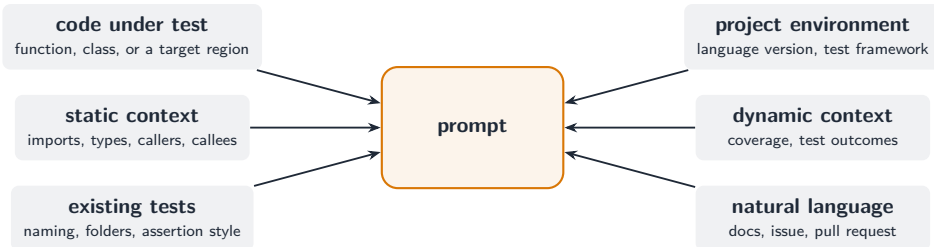
Prompt Ingredients (5/6)

The prompt decides how effective an LLM-based tool is



Prompt Ingredients (6/6)

The prompt decides how effective an LLM-based tool is



The Oracle Problem (1/4)

How do we know the **expected behavior**? Arguably the hardest problem in automated testing.

Example: a test of `price` from Pattern 1, with a bug in `discount`

```
1 // bug: discount uses qty > 10
2 Db db = test_db(); // id 7: 10 each
3 int r = price(db, 7, 10);
```

Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

The Oracle Problem (2/4)

How do we know the **expected behavior**? Arguably the hardest problem in automated testing.

- **Generic oracles**: no crash, no exception, no sanitizer report

Example: a test of `price` from Pattern 1, with a bug in `discount`

```
1 // bug: discount uses qty > 10
2 Db db = test_db(); // id 7: 10 each
3 int r = price(db, 7, 10);
```

generic: line 3 must run without a crash

The Oracle Problem (3/4)

How do we know the **expected behavior**? Arguably the hardest problem in automated testing.

- **Generic oracles**: no crash, no exception, no sanitizer report
- **Regression oracles**: assert what the code does today; catch later changes

Example: a test of `price` from Pattern 1, with a bug in `discount`

```
1 // bug: discount uses qty > 10
2 Db db = test_db(); // id 7: 10 each
3 int r = price(db, 7, 10);

5 assert(r == 100); // today
```

regression: passes today, so it keeps the bug

The Oracle Problem (4/4)

How do we know the **expected behavior**? Arguably the hardest problem in automated testing.

- **Generic oracles**: no crash, no exception, no sanitizer report
- **Regression oracles**: assert what the code does today; catch later changes
- **Bug-reproducing tests**: must *fail*, and for the *right reason*

Example: a test of `price` from Pattern 1, with a bug in `discount`

```
1 // bug: discount uses qty > 10
2 Db db = test_db(); // id 7: 10 each
3 int r = price(db, 7, 10);

5 assert(r == 100); // today
6 assert(r == 85);  // intended
```

bug-reproducing: fails today, because of the bug

LLM Oracles: The Study (1/4)

How good are oracles written by LLMs, on code they **cannot have seen**?



Molinelli et al., ASE 2025

LLM Oracles: The Study (2/4)

How good are oracles written by LLMs, on code they **cannot have seen**?



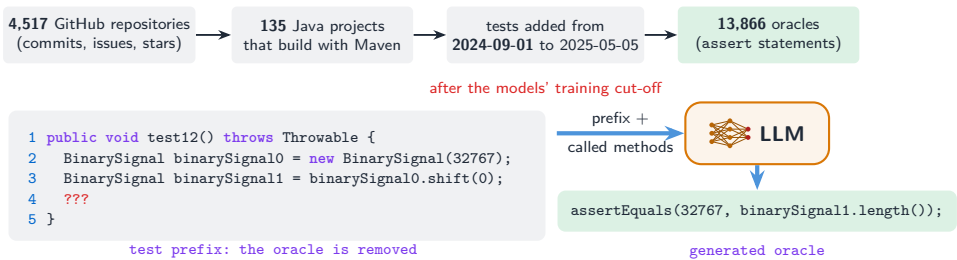

```
1 public void test12() throws Throwable {
2   BinarySignal binarySignal0 = new BinarySignal(32767);
3   BinarySignal binarySignal1 = binarySignal0.shift(0);
4   ???
5 }
```

test prefix: the oracle is removed

Molinelli et al., ASE 2025

LLM Oracles: The Study (3/4)

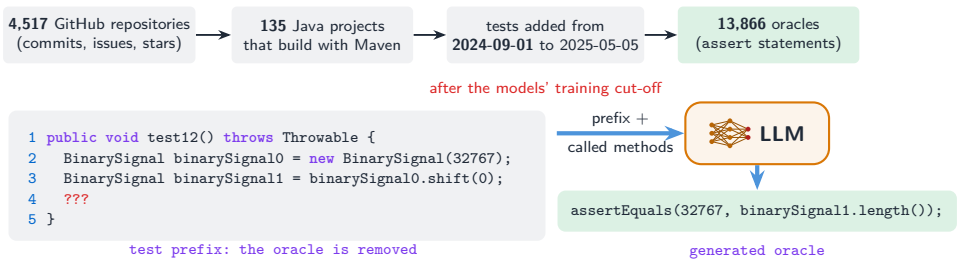
How good are oracles written by LLMs, on code they **cannot have seen**?



Molinelli et al., ASE 2025

LLM Oracles: The Study (4/4)

How good are oracles written by LLMs, on code they **cannot have seen**?



10 LLMs (Qwen2.5, Phi-4, Llama 3.3; 1.5B to 70B parameters) $\times$ 4 prompts: 610,104 queries

Molinelli et al., ASE 2025

LLM Oracles: Matching the Developer (1/5)

Does the generated `assert` **match** the one the developer wrote?

developer wrote	`assertEquals(32767, binarySignal1.length());`	
LLM, case 1	`assertEquals(32767, binarySignal1.length());`	**exact match**: the same text
LLM, case 2	`assertEquals(32767, binarySignal1.length())`	**inclusion match**: one contains the other
LLM, case 3	`assertTrue(binarySignal1.length() == 32767);`	**no match**, although correct

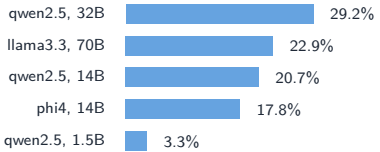
Molinelli et al., ASE 2025

LLM Oracles: Matching the Developer (2/5)

Does the generated `assert` **match** the one the developer wrote?

developer wrote	`assertEquals(32767, binarySignal1.length());`	
LLM, case 1	`assertEquals(32767, binarySignal1.length());`	**exact match**: the same text
LLM, case 2	`assertEquals(32767, binarySignal1.length())`	**inclusion match**: one contains the other
LLM, case 3	`assertTrue(binarySignal1.length() == 32767);`	**no match**, although correct

inclusion match per model, mean over its 4 prompts



e.g. qwen2.5, 32B: prompts give 28.5, 29.5, 29.0, 29.7%
mean $= 116.7/4 \approx 29.2\%$

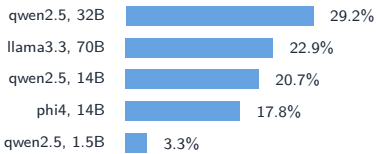
Molinelli et al., ASE 2025

LLM Oracles: Matching the Developer (3/5)

Does the generated `assert` **match** the one the developer wrote?

developer wrote	`assertEquals(32767, binarySignal1.length());`	
LLM, case 1	`assertEquals(32767, binarySignal1.length());`	**exact match**: the same text
LLM, case 2	`assertEquals(32767, binarySignal1.length())`	**inclusion match**: one contains the other
LLM, case 3	`assertTrue(binarySignal1.length() == 32767);`	**no match**, although correct

inclusion match per model, mean over its 4 prompts



e.g. qwen2.5, 32B: prompts give 28.5, 29.5, 29.0, 29.7%
mean $= 116.7/4 \approx 29.2\%$

- **Size matters**: 1.5B to 32B, from 3% to 29%

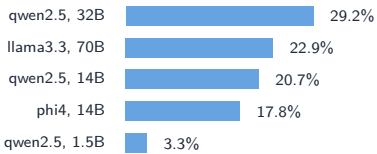
Molinelli et al., ASE 2025

LLM Oracles: Matching the Developer (4/5)

Does the generated `assert` **match** the one the developer wrote?

developer wrote	`assertEquals(32767, binarySignal1.length());`	
LLM, case 1	`assertEquals(32767, binarySignal1.length());`	**exact match**: the same text
LLM, case 2	`assertEquals(32767, binarySignal1.length())`	**inclusion match**: one contains the other
LLM, case 3	`assertTrue(binarySignal1.length() == 32767);`	**no match**, although correct

inclusion match per model, mean over its 4 prompts



e.g. qwen2.5, 32B: prompts give 28.5, 29.5, 29.0, 29.7%
mean $= 116.7/4 \approx 29.2\%$

- **Size matters**: 1.5B to 32B, from 3% to 29%
- code models do no better than general ones

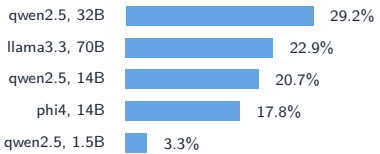
Molinelli et al., ASE 2025

LLM Oracles: Matching the Developer (5/5)

Does the generated `assert` **match** the one the developer wrote?

developer wrote	`assertEquals(32767, binarySignal1.length());`	
LLM, case 1	`assertEquals(32767, binarySignal1.length());`	**exact match**: the same text
LLM, case 2	`assertEquals(32767, binarySignal1.length())`	**inclusion match**: one contains the other
LLM, case 3	`assertTrue(binarySignal1.length() == 32767);`	**no match**, although correct

inclusion match per model, mean over its 4 prompts



e.g. qwen2.5, 32B: prompts give 28.5, 29.5, 29.0, 29.7%
mean $= 116.7/4 \approx 29.2\%$

- **Size matters**: 1.5B to 32B, from 3% to 29%
- code models do no better than general ones
- more context (test class, class under test) gives **no gain**

Molinelli et al., ASE 2025

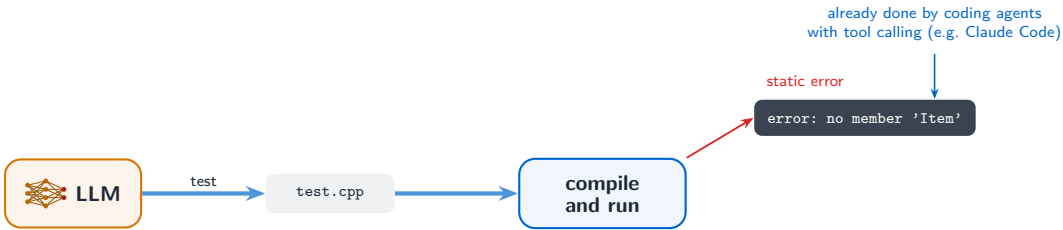
Guidance via Feedback (1/6)

Single-shot generation is limited: run the test, and **feed the result back**



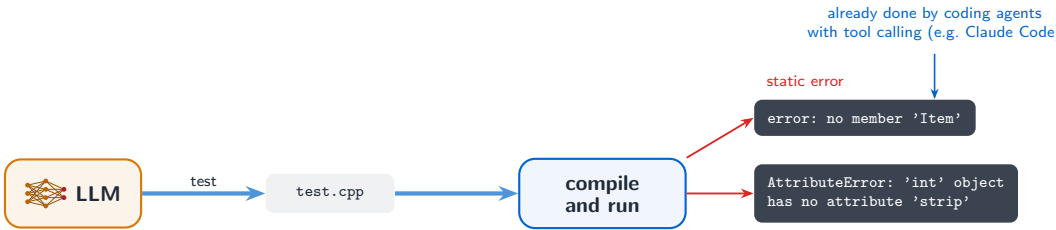
Guidance via Feedback (2/6)

Single-shot generation is limited: run the test, and **feed the result back**



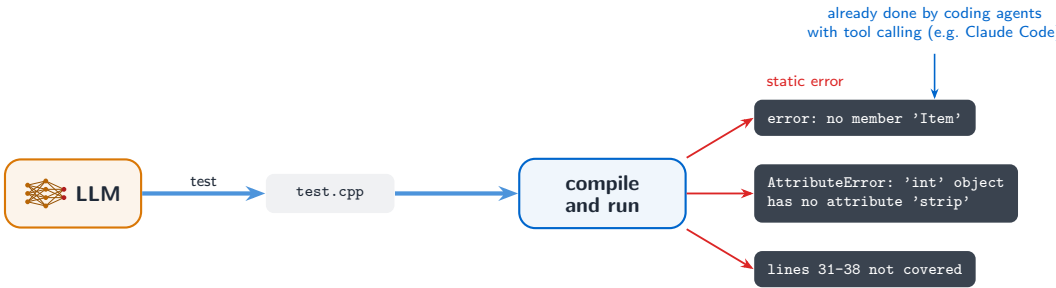
Guidance via Feedback (3/6)

Single-shot generation is limited: run the test, and **feed the result back**



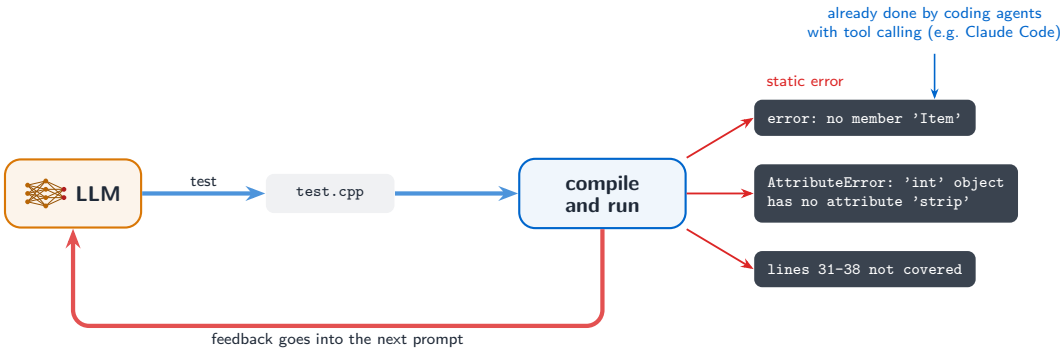
Guidance via Feedback (4/6)

Single-shot generation is limited: run the test, and **feed the result back**



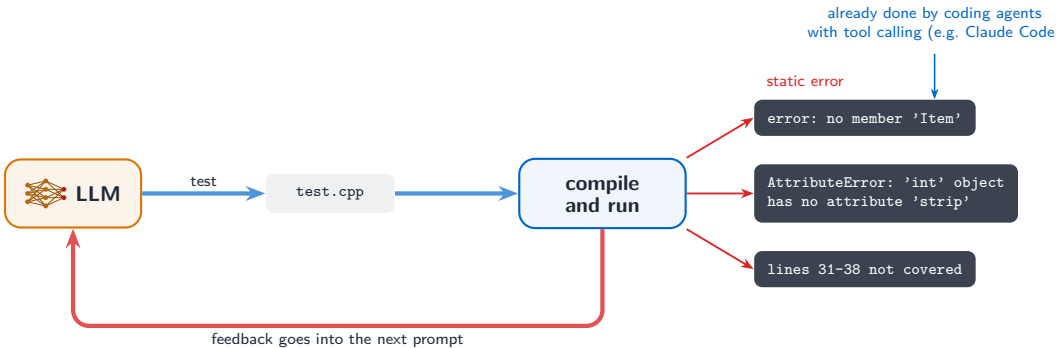
Guidance via Feedback (5/6)

Single-shot generation is limited: run the test, and **feed the result back**



Guidance via Feedback (6/6)

Single-shot generation is limited: run the test, and **feed the result back**



A failing *assertion* may be intended (bug-reproducing test) or not (regression test): the goal decides.

Quiz: Context and Feedback (1/4)

An LLM test generator has these problems. What would you add?

1. Tests call `Cart::addItem`, but the method is called `add`

2. Tests use JUnit 4, the project uses JUnit 5

3. Tests pass, but never reach the discount code

Quiz: Context and Feedback (2/4)

An LLM test generator has these problems. What would you add?

1. Tests call `Cart::addItem`, but the method is called `add` — `symbols + compiler`

2. Tests use JUnit 4, the project uses JUnit 5 —

3. Tests pass, but never reach the discount code —

Quiz: Context and Feedback (3/4)

An LLM test generator has these problems. What would you add?

1. Tests call `Cart::addItem`, but the method is called `add` — `symbols + compiler`

2. Tests use JUnit 4, the project uses JUnit 5 — `env + example tests`

3. Tests pass, but never reach the discount code

Quiz: Context and Feedback (4/4)

An LLM test generator has these problems. What would you add?

1. Tests call `Cart::addItem`, but the method is called `add` — `symbols + compiler`

2. Tests use JUnit 4, the project uses JUnit 5 — `env + example tests`

3. Tests pass, but never reach the discount code — `coverage`

Case Study: TestPilot (1/5)

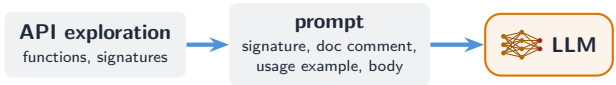
Generate unit tests for the API of an npm package (JavaScript), with Mocha assertions

API exploration
functions, signatures

Schäfer, Nadi, Eghbali, Tip, IEEE TSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Case Study: TestPilot (2/5)

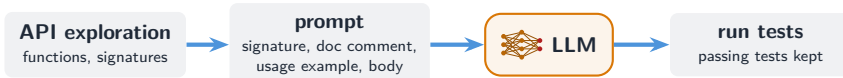
Generate unit tests for the API of an npm package (JavaScript), with Mocha assertions



Schäfer, Nadi, Eghbali, Tip, IEEE TSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Case Study: TestPilot (3/5)

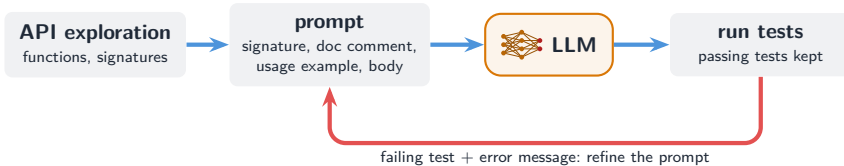
Generate unit tests for the API of an npm package (JavaScript), with Mocha assertions



Schäfer, Nadi, Eghbali, Tip, IEEE TSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Case Study: TestPilot (4/5)

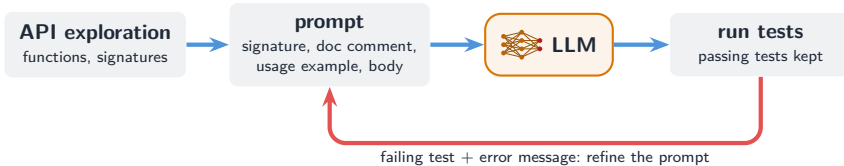
Generate unit tests for the API of an npm package (JavaScript), with Mocha assertions



Schäfer, Nadi, Eghbali, Tip, IEEE TSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Case Study: TestPilot (5/5)

Generate unit tests for the API of an npm package (JavaScript), with Mocha assertions



Example: a test expects a short country name; the library returns `"United States of America"`.
The test fails. With a **usage example** from the docs in the prompt, the next test asserts the right value.

Schäfer, Nadi, Eghbali, Tip, IEEE TSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

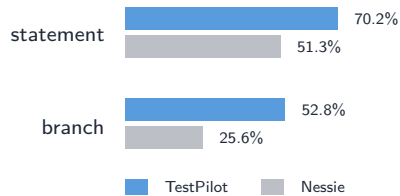
TestPilot: Results (1/3)

- 25 npm packages, 1,684 API functions
- Model: gpt-3.5-turbo

Schäfer, Nadi, Eghbali, Tip, IEEE TSE 2024

TestPilot: Results (2/3)

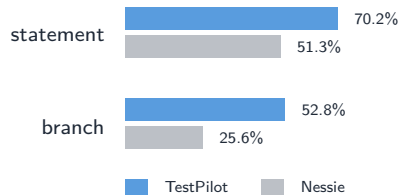
- 25 npm packages, 1,684 API functions
- Model: gpt-3.5-turbo
- Median coverage, against Nessie (a feedback-directed random generator)



Schäfer, Nadi, Eghbali, Tip, IEEE TSE 2024

TestPilot: Results (3/3)

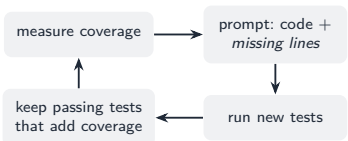
- 25 npm packages, 1,684 API functions
- Model: gpt-3.5-turbo
- Median coverage, against Nessie (a feedback-directed random generator)
- 92.8% of the generated tests have at most 50% similarity with existing tests



Schäfer, Nadi, Eghbali, Tip, IEEE TSE 2024

Coverage-Guided Generation (1/2)

CoverUp (FSE 2025, Python)

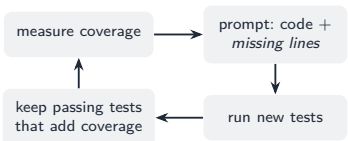


median line+branch coverage per module:
80% vs 47% (CodaMosa)

Altmayer Pizzorno and Berger, FSE 2025; Gu, Nashid, Mesbah, ICSE 2026

Coverage-Guided Generation (2/2)

CoverUp (FSE 2025, Python)



median line+branch coverage per module:
80% vs 47% (CodaMosa)

Panta (ICSE 2026, Java)

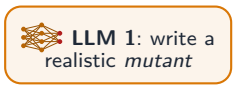
Static analysis lists the *paths* of the control-flow graph; dynamic coverage shows which are missed; each round, the LLM targets the missed paths.

+26% line and **+23%** branch coverage over the state of the art

Altmayer Pizzorno and Berger, FSE 2025; Gu, Nashid, Mesbah, ICSE 2026

Mutation-Guided Tests (1/5)

Coverage is not the goal: tests should catch **the faults that matter**, here privacy faults

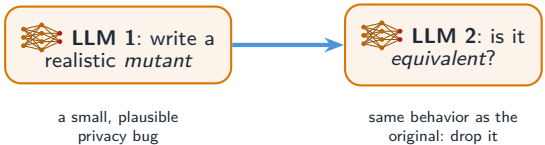


a small, plausible privacy bug

Harman et al., FSE 2025 Industry

Mutation-Guided Tests (2/5)

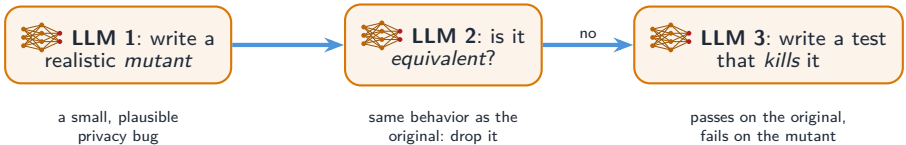
Coverage is not the goal: tests should catch **the faults that matter**, here privacy faults



Harman et al., FSE 2025 Industry

Mutation-Guided Tests (3/5)

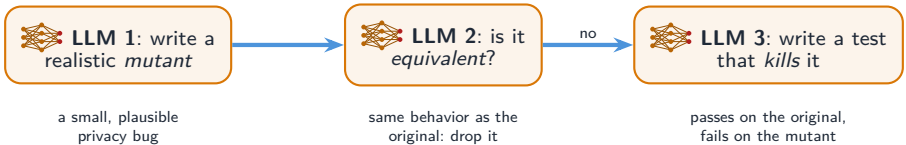
Coverage is not the goal: tests should catch **the faults that matter**, here privacy faults



Harman et al., FSE 2025 Industry

Mutation-Guided Tests (4/5)

Coverage is not the goal: tests should catch **the faults that matter**, here privacy faults



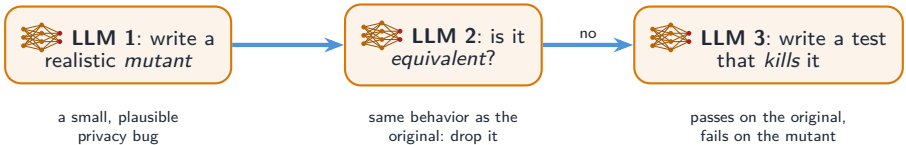
Results at Meta



Harman et al., FSE 2025 Industry

Mutation-Guided Tests (5/5)

Coverage is not the goal: tests should catch **the faults that matter**, here privacy faults



Results at Meta



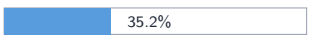
Engineers judged about 36% of the reviewed tests possibly or definitely related to privacy

Harman et al., FSE 2025 Industry

How Good Are LLM-Generated Tests? (1/3)

TestGenEval (ICLR 2025)

68,647 tests from 1,210 code and test file pairs,
11 Python repositories



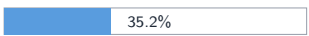
average coverage of the best model, GPT-4o

Jain, Synnaeve, Rozière, ICLR 2025; Ouédraogo et al., EMSE 2026

How Good Are LLM-Generated Tests? (2/3)

TestGenEval (ICLR 2025)

68,647 tests from 1,210 code and test file pairs, 11 Python repositories



average coverage of the best model, GPT-4o

Prompting study (EMSE 2026)

4 models, 5 prompting styles, against EvoSuite, 216,300 test cases

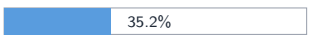
hallucinated symbols persist:
compilation failure rates reaching 86%

Jain, Synnaeve, Rozière, ICLR 2025; Ouédraogo et al., EMSE 2026

How Good Are LLM-Generated Tests? (3/3)

TestGenEval (ICLR 2025)

68,647 tests from 1,210 code and test file pairs, 11 Python repositories



average coverage of the best model, GPT-4o

Prompting study (EMSE 2026)

4 models, 5 prompting styles, against EvoSuite, 216,300 test cases

hallucinated symbols persist:
compilation failure rates reaching 86%

On real repositories, naive generation is weak: context and feedback are essential

Jain, Synnaeve, Rozière, ICLR 2025; Ouédraogo et al., EMSE 2026

Limitations (1/4)

- **Hallucinations**: the LLM invents functions and classes

Limitations (2/4)

- **Hallucinations**: the LLM invents functions and classes
- **Unrealistic tests**
  - over-mocking: the test checks the mocks, not the code
  - not integrated into the existing test suite

Pradel, Program Analysis, Stuttgart: LLM-based Test Generation; Molinelli et al., ASE 2025

Limitations (3/4)

- **Hallucinations**: the LLM invents functions and classes
- **Unrealistic tests**
  - over-mocking: the test checks the mocks, not the code
  - not integrated into the existing test suite
- **Cost**: each test needs one or more LLM calls

Pradel, Program Analysis, Stuttgart: LLM-based Test Generation; Molinelli et al., ASE 2025

Limitations (4/4)

- **Hallucinations**: the LLM invents functions and classes
- **Unrealistic tests**
  - over-mocking: the test checks the mocks, not the code
  - not integrated into the existing test suite
- **Cost**: each test needs one or more LLM calls
- **Data leakage**: the model may have seen the benchmark in training

Pradel, Program Analysis, Stuttgart: LLM-based Test Generation; Molinelli et al., ASE 2025